

¿Por qué importan las mónadas?

Antonio Locascio

25 de noviembre de 2020

1. Introducción a mónadas 2: Electric Boogaloo

Robando una frase sobre Teoría de Categorías, la mejor introducción a mónadas es la segunda (o tercera) que uno lee. Debe haber pocos temas en la computación a los que se les dediquen tantos tutoriales¹ y explicaciones como este mismo. Algunos se basan en ejemplos, otros encaran el tema mediante definiciones matemáticas y unos tantos buscan reducir el concepto con alguna analogía frecuentemente opaca². En esta introducción se toma otra alternativa³: En lugar de responder la pregunta *¿Qué es una mónada?* se responderá (o al menos se va a intentar responder) a *¿Por qué usamos mónadas?* y *¿Por qué importan en la programación funcional?*. A partir de estas preguntas se alcanzarán las definiciones que ya conocen, pero quizás con un poco más de motivación detrás de ellas.

2. ¿Por qué importa la programación funcional?

Para empezar, demos unos pasos atrás y consideremos por qué en los últimos meses le estuvimos dedicando tanta atención a la programación funcional. Es evidente que la posibilidad de modularizar un programa es fundamental para el buen diseño de software.

En su famoso artículo [1] homónimo a esta sección, John Hughes destaca que las maneras en las cuales se puede sub-dividir un programa dependen de manera directa de los *pegamentos* que permitan unir las respectivas sub-soluciones. Por lo tanto, para aumentar su capacidad de modularización, un lenguaje debe proveer nuevos pegamentos para unir programas.

Hughes reconoce que los lenguajes funcionales introducen dos nuevos pegamentos muy importantes. El primero corresponde a las funciones de alto orden, que permiten combinar funciones que abstraen ciertos comportamientos frecuentes con funciones que concretizan el uso de las anteriores. El segundo, sobre el cual se centra el resto de este documento, corresponde a la composición de funciones, que permite pegar dos programas de tal manera que la salida de uno sea la entrada del otro.

3. Composición de funciones

Repasemos un poco sobre lo que conocemos sobre la composición. Sean $f : A \rightarrow B$ y $g : B \rightarrow C$. La composición de estas funciones corresponde a la función $(g \circ f) : A \rightarrow C$. Gráficamente,

$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ & \searrow g \circ f & \downarrow g \\ & & C \end{array}$$

Notación 1 Se escribe $A \xrightarrow{f} B$ para representar una función $f : A \rightarrow B$.

Ahora bien, además de su definición, el operador de composición de funciones cuenta con las siguientes propiedades:

¹Tan solo con visitar https://wiki.haskell.org/Monad_tutorials_timeline puede darse una idea.

²Fenómeno conocido como la *Falacia del tutorial de mónadas*.

³Seguiendo muchas otras explicaciones que eligieron este camino, esto no es más que otra permutación de las explicaciones anteriores.

1. Para todo A , existe una función $id : A \rightarrow A$ tal que:

- a) Para toda $f : A \rightarrow B$, $f \circ id = f$, y
- b) Para toda $g : B \rightarrow A$, $id \circ g = g$.

Es decir, id es neutro a derecha e izquierda de $\circ$.

2. Dadas tres funciones $f : A \rightarrow B$, $g : B \rightarrow C$ y $h : C \rightarrow D$, vale que $(h \circ g) \circ f = h \circ (g \circ f)$. Es decir, $\circ$ es asociativo.

Estas propiedades garantizan que la composición se comporte como esperamos.

4. Programas con efectos

Las ventajas de programar mediante funciones matemáticas *puras* tienen un costo: un programa no puede tener efectos secundarios arbitrarios. Como se vio en la materia, para hacer frente a este problema, se usan funciones del tipo $f : A \rightarrow T B$, donde T captura los efectos que puede generar una invocación de f .

Algunos ejemplos básicos de $T B$ son:

- **Maybe** B : para programas que pueden fallar.
- **Either** $E B$: para programas que pueden lanzar errores de tipo E .
- $S \rightarrow (S, B)$: para programas con estado de tipo S .

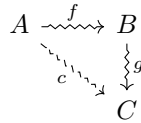
Teniendo esto en cuenta, vamos a abstraernos del efecto modelado en sí, considerando un efecto genérico T . Adicionalmente, se llama T -programa a una función $f : A \rightarrow T B$.

Notación 2 Dado un efecto T , se notará como $f : A \rightsquigarrow B$ a un T -programa $f : A \rightarrow T B$. Esta notación nos permite olvidarnos temporalmente del efecto T causado por f .

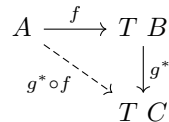
5. Composición de programas con efectos

Ahora que contamos con una forma de modelar programas con efectos, queremos recuperar el *pagamento* de programas que teníamos para funciones puras. Para ello, se va a necesitar una forma de componer T -programas.

Sean $f : A \rightsquigarrow B$ y $g : B \rightsquigarrow C$, ¿cómo se compone g con f ? Gráficamente,



¿Cómo definimos la función c ? El problema se encuentra en que realmente contamos con las funciones $f : A \rightarrow T B$ y $g : B \rightarrow T C$, cuyos tipos no son compatibles. Para poder componerlas, necesitamos una función $g^* : T B \rightarrow T C$.



Con ella, se podría definir la composición $(g^* \circ f) : A \rightsquigarrow C$, que es lo que buscábamos. Por lo tanto, si para toda función $g : B \rightarrow T C$ podemos definir una función $g^* : T B \rightarrow T C$, esta composición estará bien definida. Visto de otro modo, lo que se necesita realmente no es más que un operador $(*) : (B \rightarrow T C) \rightarrow (T B \rightarrow T C)$ que dada una g retorne la g^* adecuada.

En base a este operador, la composición $(g^* \circ f)$ puede verse de dos maneras. La primera es como la composición usual de la función f con la función g^* . La segunda, quizás más interesante,

como una nueva operación de composición “ $\circ$ ”, que permite componer los T -programas f y g . Notaremos con “ $\circ$ ” a esta noción, que formalmente sería:

$$\begin{aligned} (*\circ) &: (B \rightarrow T C) \rightarrow (A \rightarrow T B) \rightarrow A \rightarrow T C \\ * \circ &= \lambda g f. (g^*) \circ f \end{aligned}$$

Si bien el operador $(*)$ permite definir la composición de dos T -programas, todavía falta establecer que esta se comporte de la manera esperada. Para ello, es necesario volver a enunciar las propiedades de la composición vistas en la Sección 3. Estas leyes, que aseguran que esta nueva noción de composición es correcta, son:

1. Para todo A , existe una función $id : A \rightsquigarrow A$ tal que:

- a) Para toda $f : A \rightsquigarrow B$, $f^* \circ id = f$, y
- b) Para toda $g : B \rightsquigarrow A$, $id^* \circ g = g$.

Es decir, id es neutro a derecha e izquierda de “ $\circ$ ”.

2. Dadas tres funciones $f : A \rightsquigarrow B$, $g : B \rightsquigarrow C$ y $h : C \rightsquigarrow D$, vale que $(h^* \circ g)^* \circ f = h^* \circ (g^* \circ f)$. Es decir, “ $\circ$ ” es asociativo.

En resumen, para poder componer T -programas se requiere un **operador** $(*) : (A \rightarrow T B) \rightarrow (T A \rightarrow T B)$ y una **función** $id : A \rightsquigarrow A$ tales que esta última sea **elemento neutro** de la composición dada por $(*)$ y esta noción de composición sea **asociativa**.

6. ¿Y las mónadas?

Volvamos a ver lo que pedimos, prestandole atención a sus tipos. En primer lugar requerimos el operador $(*)$ de tipo

$$(A \rightarrow T B) \rightarrow (T A \rightarrow T B)$$

Ahora bien, por asociatividad de $\rightarrow$, esto no es más que

$$(A \rightarrow T B) \rightarrow T A \rightarrow T B$$

Visto de este modo, se puede pensar que es una función que toma dos argumentos, una función y un $T A$. ¿Qué pasa si invertimos su orden?

$$T A \rightarrow (A \rightarrow T B) \rightarrow T B$$

¿Les parece familiar este tipo? Efectivamente, el operador $(*)$ no es otra cosa que nuestro conocido $\gg$ con el orden de sus argumentos invertidos. En efecto,

$$(*) f m = m \gg f \tag{6.1}$$

Comentario 1 El operador $(*)$ existe en Haskell, bajo el nombre de $\gg$.

Ahora pongamos el foco en la función id , cuyo tipo es

$$A \rightsquigarrow A$$

O, equivalentemente,

$$A \rightarrow T A$$

Como seguramente adivinaron, la función id no es más que la función `return` de la definición de mónadas vista en clase:

$$id = \text{return} \tag{6.2}$$

En este punto lo único que falta para llegar a la definición de mónadas son las leyes monádicas. Pero todavía contamos con las propiedades de la sección anterior, veamos que pasa si pasamos de $(*)$ e id a $\gg$ y `return`.

Arrancamos con la propiedad (1.a), $f^* \circ id = f$, manipulando su lado izquierdo al aplicar una variable x :

$$\begin{aligned}
& (f^* \circ id) \ x \\
= & \{\text{Def. } \circ\} \\
& (f^*) (id \ x) \\
= & \\
& (*) \ f \ (id \ x) \\
= & \{\text{Prop. 6.1}\} \\
& (id \ x) \gg= f \\
= & \{\text{Prop. 6.2}\} \\
& (\text{return } x) \gg= f
\end{aligned}$$

Luego, al reemplazar esta equivalencia en la propiedad (1.a), se obtiene la ley (*monad.1*):

$$(\text{return } x) \gg= f = f \ x$$

Procediendo de manera análoga, se puede ver que las propiedades (1.b) y (2) son equivalentes a (*monad.2*) y (*monad.3*) respectivamente. Se deja como ejercicio verificarlo.

Comentario 2 La composición de T -programas $(*)$ se conoce como composición de Kleisli y en Haskell tiene el nombre $\gg$ ($\gg=$).

7. Conclusión

Repasemos brevemente lo visto. Empezamos destacando la importancia de la composición de funciones como herramienta para modularizar programas. Luego, nos encontramos con la necesidad de modelar efectos secundarios causados por un programa mediante un constructor de tipos T , por lo que se pierde la capacidad de componer programas. Para recuperarla, tuvimos que introducir un operador sobre programas $(*)$ junto a un programa id , que cumplen ciertas propiedades que garantizan que estos definen correctamente la composición de programas con efectos T . Por último, mostramos que estos requerimientos no eran más que los de la definición de mónadas vista en clase.

De esta manera, entonces, se puede apreciar que una mónada no es más que la estructura necesaria para componer programas con efectos, y que en esto yace su importancia en la programación funcional.

Referencias

- [1] J. Hughes. “Why Functional Programming Matters”. En: *Comput. J.* 32.2 (abr. de 1989), págs. 98-107. ISSN: 0010-4620. DOI: [10.1093/comjnl/32.2.98](https://doi.org/10.1093/comjnl/32.2.98).
- [2] Eugenio Moggi. “Notions of computation and monads”. En: *Information and Computation* 93.1 (1991). Selections from 1989 IEEE Symposium on Logic in Computer Science, págs. 55-92. ISSN: 0890-5401. DOI: [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4).
- [3] Emily Riehl. “A categorical view of computational effects”. Compose Conference. 2017. URL: <http://www.math.jhu.edu/~eriehl/compose.pdf>.